

# Analysis Pipeline — Conversion Equations

Documents every transform in **modFunc.py** that turns a raw or averaged **.dat** value into something else. **time[1]–time[0]** is the raw MCell output timestep (**STEP = 10\*dt** in **core/rxn\_outputRS.mdl**); **step** in the equations below is each method's own averaging/differencing window — a separate parameter from the raw timestep.

| Transform                          | Method                               | Input → Output                                                                                                   | Equation                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seed averaging                     | avg_dat                              | per-seed dat/<file> (all seeds) → averaged <file>                                                                | $\text{avg}[t] = (1/j) \cdot \sum_i \text{raw}_i[t]$ j = number of seeds successfully averaged (a seed is <i>skipped</i> , not zero-filled, if its row count doesn't match the first seed's)                                                                                                                                                                                                                                          |
| Cumulative-average → instantaneous | computeCaConc (via conc_calc)        | Ca.Conc.az (ca.dat col 1) → CaConc                                                                               | $c\_tc = \text{time} \times \text{Ca.Conc.az}$ $\text{conc}[i] = (c\_tc[i+\text{step}] - c\_tc[i]) / dt, dt = \text{step} \cdot (\text{time}[1] - \text{time}[0]), \text{ default step} = 5$ <i>Ca.Conc.az is MCell's ESTIMATE_CONCENTRATION on the active_zone_plane region — a cumulative average since t=0, not instantaneous — so this finite difference of the time-weighted value recovers the instantaneous concentration.</i> |
| Count → concentration, presynaptic | pre_ca_conc                          | Ca.Pre (ca.dat col 2) → pre_ca_conc.dat                                                                          | $[\text{Ca}]_{\text{pre}} = (\text{Ca.Pre} / 6.022 \times 10^{23}) / 9.6 \times 10^{-16} \times 10^9 \text{ (nM)}$ $\text{presynaptic\_vol} = 0.961 \mu\text{m}^3 = 9.6 \times 10^{-16} \text{ L, from misc\_*.mdl (presynaptic\_vol} = 4 \cdot 0.5 \cdot 0.5 - \text{er\_vol})$                                                                                                                                                      |
| Count → concentration, ER          | er_ca_conc                           | Ca.ER (ca.dat col 3) → er_ca_conc.dat                                                                            | $[\text{Ca}]_{\text{ER}} = (\text{Ca.ER} / 6.022 \times 10^{23}) / 3.9 \times 10^{-17} \times 10^6 \text{ (}\mu\text{M)}$ $\text{er\_vol} = 0.039 \mu\text{m}^3 = 3.9 \times 10^{-17} \text{ L, from misc\_*.mdl (er\_vol} = 3.9 \cdot 0.1 \cdot 0.1)$                                                                                                                                                                                |
| Cumulative flux → rate             | fluxCurrent                          | flux file (vdcc_pq_ca_flux.dat, serca_ca_flux.dat, ryr_ca_flux.dat) → *Rate.dat                                  | $\text{rate}[i] = \text{ncharge} \cdot \text{charge} \cdot (\text{flux}[i+\text{step}] - \text{flux}[i]) / dt$ $dt = \text{step} \cdot (\text{time}[1] - \text{time}[0]), \text{ default step} = 10, \text{ ncharge} = \text{charge} = 1 \text{ (reduces to a plain finite difference)}$                                                                                                                                              |
| Release probability                | relppf (paired-pulse) relptp (train) | rel.dat event timestamps (concatenated vdcc.sync_*/vdcc.async_*/vdcc.spont.dat) → result file (mean row, SD row) | AP times: $ts_i = (i \cdot \text{isi} + 2.0) / 1000 \text{ s (relppf)}$ , or $(i \cdot \text{isi}) / 1000 \text{ s (relptp, no 2ms offset)}$<br>trial counted "released" at pulse i if any event $\in (ts_i, ts_i + 0.02)$<br>$Pr_i = n\text{Rel}_i / n\_dirs$ , over 1000 bootstrap resamples (seed dirs, with replacement) → mean & SD<br>relppf also reports facilitation $Pr_i / Pr_0$ for $i \geq 1$                             |

Source: analysis/modFunc.py, cross-checked against core/rxn\_outputRS.mdl and misc\_250.mdl / misc\_750.mdl (AD-presynaptic-calcium-model repo).